Short Introduction to SCM, Git and GitHub

D Rodriguez, D F Barrero

University of Alcala, UAH, Spain

Objectives and References

Objectives

1. Understand why software projects need source control.
2. Learn the basic Git workflow for day-to-day development.
3. Identify the main collaboration features provided by GitHub.
4. Understand the basics of CI/CD and security analysis in GitHub.

References

1. Git documentation: `https://git-scm.com/doc`
2. GitHub Docs: `https://docs.github.com/`
3. GitHub Skills: `https://skills.github.com/`
4. Interactive tutorial: `https://learngitbranching.js.org/`

Table of Contents

1. Software Configuration Management
   - Motivation
   - Version Control
   - Source Configuration Management
2. Git
   - What is Git?
   - Git Hosting Sites
   - Git in IDEs
   - Git vs. SVN
   - Repositories
   - Workflow Overview
3. Using Git
   - Basic Workflow
   - Initializing a Repository
   - Basic Commands
   - Commits
   - Merge and Conflicts
   - Branches
   - Tags
   - Advanced Git
   - Good Practices
4. GitHub
   - Features
   - Automation
   - Security
   - Repository Creation
   - README
   - Markdown
   - GitHub Pages

Software Configuration Management

Motivation

- Multiple developers edit the same codebase over time.
- Files evolve, bugs are fixed, and features are added incrementally.
- Teams need to know who changed what, when, and why.
- Reverting a broken change should be routine, not a crisis.

Without source control, collaboration becomes fragile and error-prone.

Version control systems

Version control systems keep track of changes to source code and project files. They allow multiple people to edit a project in a predictable manner.

- They store history.
- They help coordinate collaborative work.
- They make rollback and auditing possible.

Software Configuration Management

Source Configuration Management

Software configuration management is the task of tracking and controlling changes in software artifacts, and it is part of the broader field of configuration management.

Main open-source version control systems:

- 1982: RCS
- 1990: CVS
- 2000: Subversion
- 2005: Git and Mercurial

There are many proprietary alternatives, but `Git` is now the dominant one.

All software should live under version control. Otherwise, maintenance becomes risky.

Git

What is Git?

Git is an open source distributed version control system created by Linus Torvalds.

`https://git-scm.com/`

Compared with centralized systems, every clone contains the full repository history.

It is usually easier to start with a hosted platform than to maintain your own server.

- GitHub: one of the most widely used Git hosting platforms; strong ecosystem and collaboration workflows.
- GitLab: integrated DevSecOps platform with strong self-managed and cloud options.
- Bitbucket: a Git hosting platform integrated with Atlassian tools such as Jira and Confluence.

Hosted platforms are commonly used as a central collaboration hub:

- from which everyone pulls other people's changes
- to which everyone pushes their own changes

Note: plans and pricing details change frequently across providers.

Git

Git Hosting Platforms at a Glance

GitHub

- Broad ecosystem and open source reach
- GitHub Actions
- Great default choice for many teams

GitLab

- Integrated DevSecOps platform
- GitLab CI/CD
- Strong self-managed options

Bitbucket

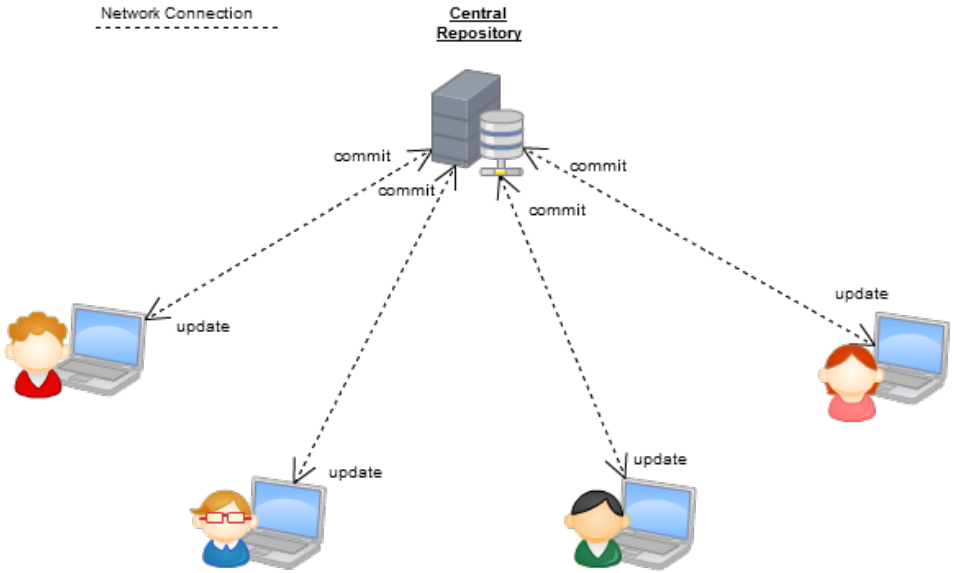
- Tight Jira and Confluence integration
- Bitbucket Pipelines
- Good fit for Atlassian-centered teams

Most modern development environments integrate Git directly.

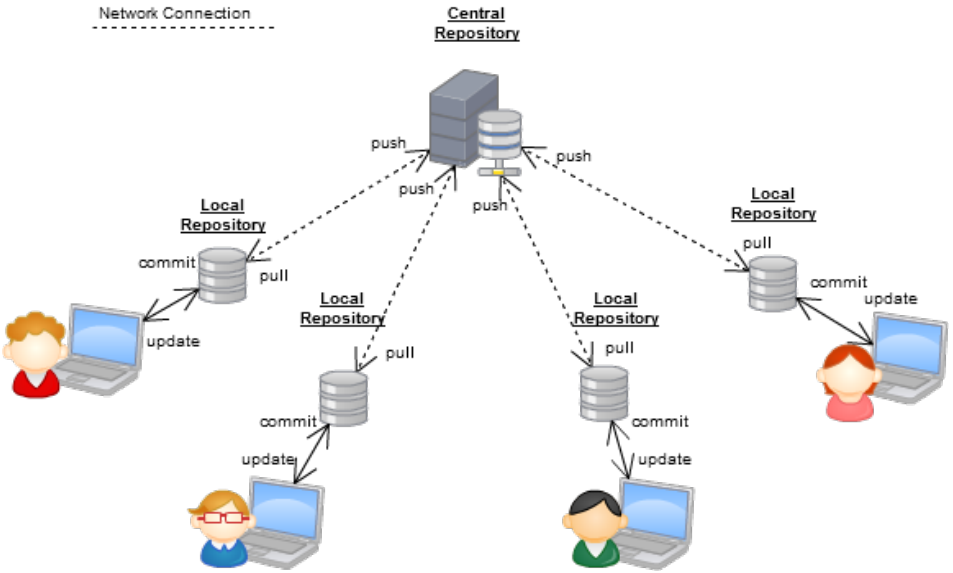
- Commit, push, pull, merge, and resolve conflicts without leaving the editor.
- Inspect file history and diffs visually.
- Review branches, stashes, tags, and remotes from a GUI.

Examples include VS Code, PyCharm, IntelliJ IDEA, Eclipse, and many more.

Centralized (SVN)

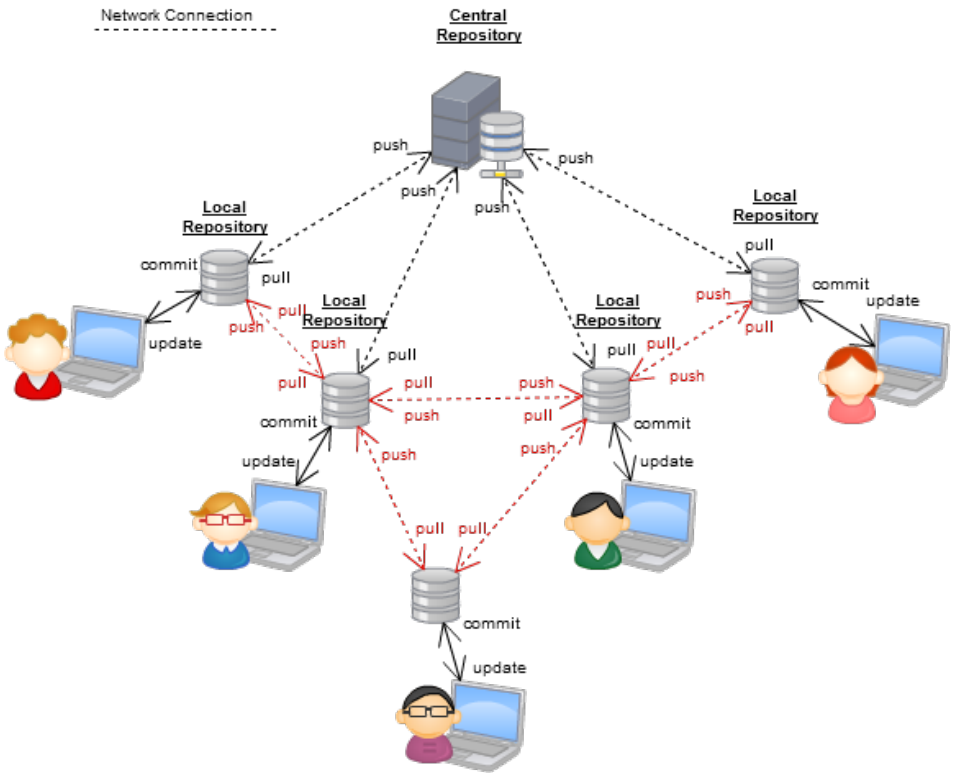


Distributed (Git)



(Source)

Fully distributed (Git)



(Source)

Git concepts to know

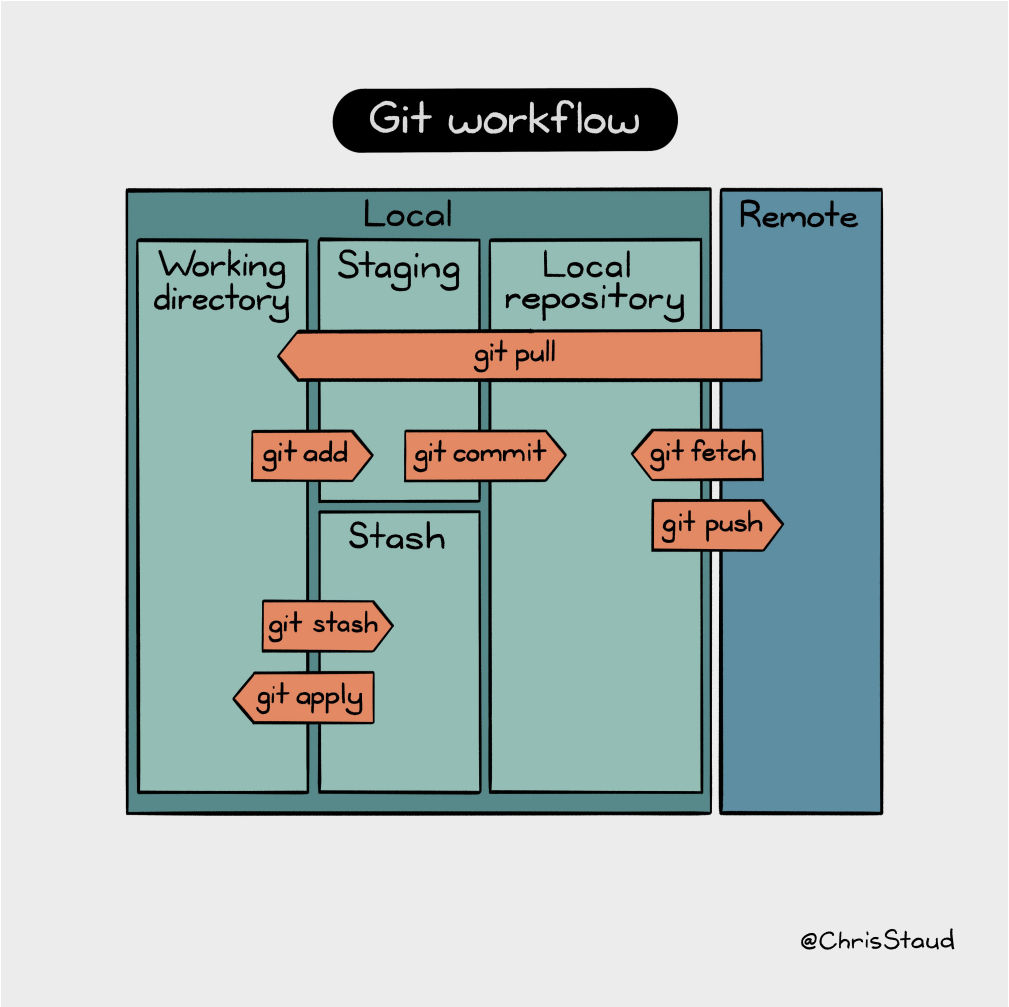
- `commit`
- `push`, `pull`, `fetch`
- `branch`, `merge`
- `origin`, `remote`

Key Git concepts to know:

- `Working directory`: your project files as they currently exist on disk.
- `Repository`: the database that stores the history of changes.
- `Local repository`: the repository in your own machine.
- `Remote repository`: a copy of the repository hosted on a server.
- `Staging area`: the intermediary space used to prepare the next commit.

Git

Workflow Overview



Given initialized local and remote repositories, a common daily workflow is:

1. Pull changes from the remote repository.
2. Edit your files locally.
3. Add the changes to the staging area.
4. Commit the staged changes.
5. Push your commits to the remote repository.

Using Git

Initializing a Repository

When using Git for the first time:

```
git config --global user.email user@example.com
git config --global user.name  "Jane Doe"
```

Initialize a local repository:

```
mkdir /path/to/your/project
cd /path/to/your/project
git init
echo "# My project" > README.md
git add README.md
git commit -m "Initial commit"
git branch -M main
git remote add origin https://<where>/<project.git>
git push -u origin main
```

- `git init` creates the hidden `.git` directory.
- All Git metadata lives inside `.git`, which makes initialization safe.
- By default, a freshly initialized repository has no remotes.

To work with an existing remote repository, the usual starting point is to clone it:

```
git clone <repo>
```

Example:

```
git clone https://github.com/danrodgar/intro-git.git
```

Once cloned, you can edit the repository locally as much as you want. No changes reach the remote until you explicitly push them.

To start tracking changes, we typically `add`, `commit`, and then `push`:

```
echo "A new file..." >> README.md
git add README.md
git commit -m "Initial commit"
git push -u origin main
```

- `git add` moves changes into the staging area.
- `git commit` records a snapshot of staged changes.
- `git push` sends your local commits to the remote repository.

Before starting new work, it is good practice to synchronize with the remote:

- `git pull` downloads and integrates new remote changes.
- If you have unfinished local modifications, `git stash` can temporarily save them.
- `git status` shows the current state of your working tree.
- `git checkout - <file>` or modern alternatives such as `git restore <file>` can discard local changes.

After committing locally, publish the result to the remote repository:

```
git push origin main
```

Pushing transfers your local commits. It does not automatically include uncommitted or unstaged changes.

Each commit has:

- an ID (its hash)
- an author
- a date
- a message describing the change

```
commit 44161dde6ea234f8cb997644f8e187123c3cc4af
Author: David <foo@foo.com>
Date:   Fri Mar 9 14:57:32 2018 +0100

    Issue with syntax highlighting solved
```

Merging is a common operation in Git.

- Changes in different parts of a file are usually merged automatically.
- Conflicts happen when Git cannot decide how to merge two changes safely.
- Human intervention is then required.

Conflict resolution

1. Identify the conflicted files.
2. Open them and choose which changes to keep.
3. Remove the conflict markers.
4. Stage the resolved files and commit.

Conflict markers

```
def greet():
<<<<<<< HEAD
    return "Hello"
=======
    return "Hola"
>>>>>>> feature
```

Using Git

Handling Conflicts in Practice

When a `git pull` reports conflicts, a common workflow is:

```
git status
# edit conflicted files and remove <<<<<<< ======= >>>>>>>
git add <resolved-file>
git commit
```

If you want to cancel the merge attempt:

```
git merge --abort
```

If you are resolving conflicts in a rebase:

```
git rebase main
# resolve files
git add <resolved-file>
git rebase --continue
```

To cancel the rebase instead:

```
git rebase --abort
```

```
diff -u <old file> <new file>
```

This command shows the changes needed to turn the old file into the new file.

- `--` and `+++` identify the old and new filenames.
- `@@` marks the location of the change.
- A leading space means the line is unchanged.
- `-` marks a deleted line.
- `+` marks an added line.

Using Git

Merge and Conflicts: diff Example

Old source file (hello.c)

```
#include <stdio.h>
int main() {
    printf("Hello World\n");
}
```

New source file (hello_new.c)

```
#include <stdio.h>
int main(int argc, char *argv[]) {
  printf("Hello World\n");
  return 0;
}
```

Command used to generate the patch

```
$ diff -u hello.c hello_new.c > hello.patch
```

Generated patch output (hello.patch)

```
--- hello.c
+++ hello_new.c
@@ -1,4 +1,5 @@
 #include <stdio.h>
-int main() {
+int main(int argc, char *argv[]) {
   printf("Hello World\n");
+  return 0;
 }
```

After generating a patch file:

```
diff -u <old file> <new file> > file.patch
```

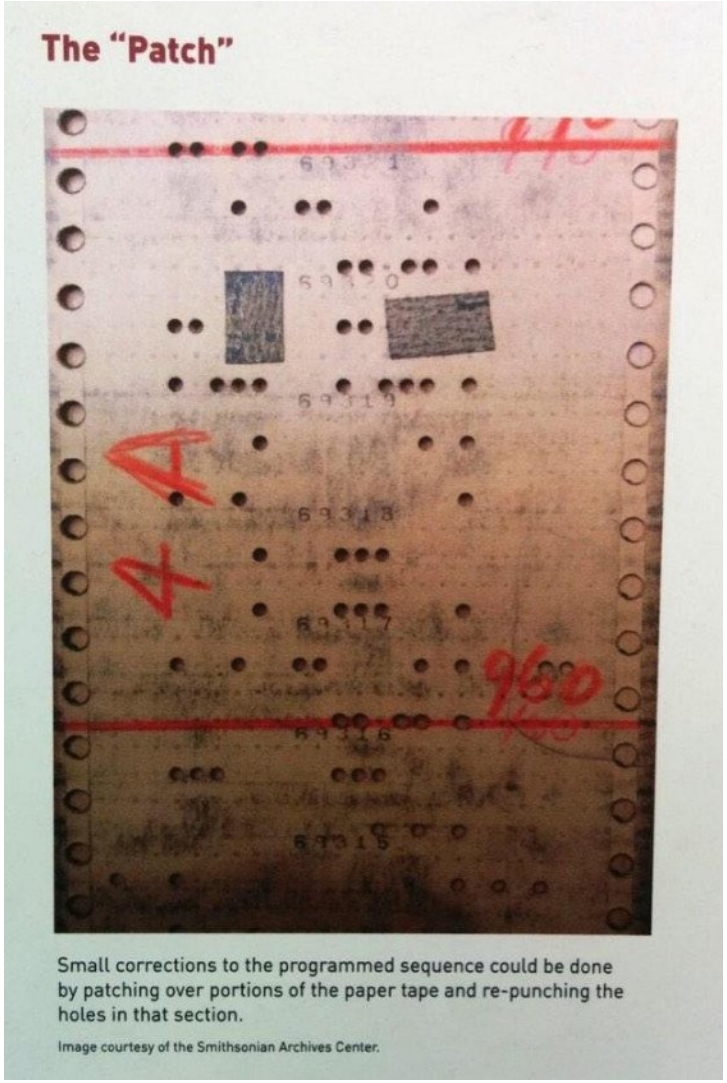
you can apply it with:

```
patch < file.patch
```

The patch file contains enough information to identify the target file and the required edits.

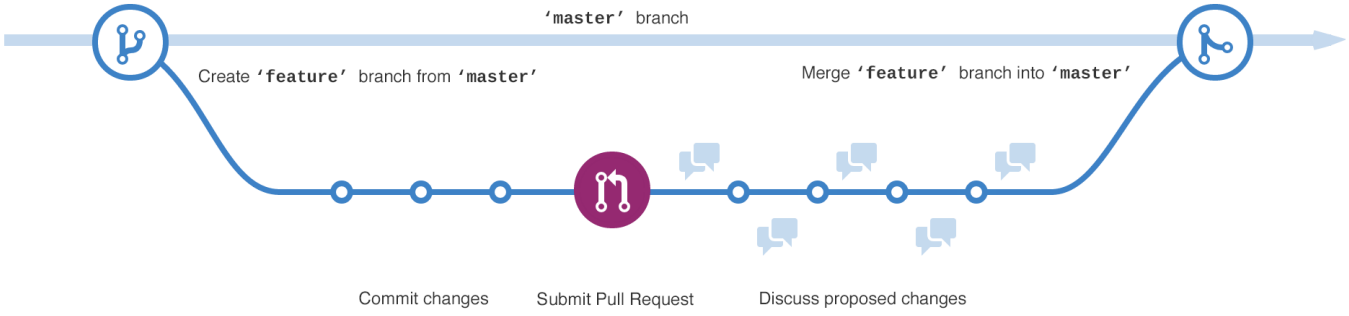
Using Git

Original Patch



Using Git

Branches



Branches are used extensively to separate lines of development.

- A repository can have multiple branches locally and remotely.
- The default branch is usually called main or master.
- `git branch <name>` creates a branch.
- `git checkout <branch>` or `git switch <branch>` changes the active branch.
- `HEAD` points to the current branch or, sometimes, to a specific commit.

A tag is a pointer to a specific point in repository history.

- Tags usually have meaningful names such as `v1.0` or `release-2026-03`.
- They are commonly used to mark releases.
- Unlike branches, tags typically do not move after they are created.

Sometimes you need to inspect an earlier state of the repository.

- `git log`
- `git log -oneline`

These commands help locate old commits that you can inspect or recover files from.

Using Git

Good Practices

Learn on the job: the best way to learn Git is by using it consistently.

Best practices:

- Regularly push and pull.
- Test before pushing.
- Do not push half-baked changes.
- Do not pull in the middle of a risky unfinished task.
- Never commit temporary or generated files unless they belong in the repository.
- Keep commit messages short and informative.
- Keep the main branch stable and functional.

Remember: Git does not overwrite local changes without an explicit command.

GitHub

Features

GitHub is a Git hosting platform with collaboration features on top of the repository itself.

- Free public and private repositories (with plan limits)
- Repository browser
- Pull requests
- Issue tracking
- Code review workflows
- CI/CD with GitHub Actions
- Security analysis (CodeQL, Dependabot)
- Markdown rendering
- Project documentation and static hosting

Key GitHub concepts to know

- `Pull request`
- `Fork`
- `Issue`
- `README.md`

- A fork creates your own copy of someone else's repository.
- A pull request proposes changes for review and discussion before merging.

Typical pull request (PR) flow:

1. Create a branch from `main` (for example, `feature/login-form`).
2. Commit focused changes and push the branch.
3. Open a PR with clear title, context, and test notes.
4. Request review and address comments with follow-up commits.
5. Keep branch updated (merge or rebase from `main`).
6. Merge when checks pass and review is approved.

Good PR habits

Small PRs, descriptive commit messages, and clear review notes speed up integration.

GitHub Actions is GitHub's native workflow automation and CI/CD system.

- Workflows are defined as YAML files in `.github/workflows`.
- Typical triggers: push, pull request, schedule, manual dispatch.
- Jobs can run tests, build artifacts, publish releases, or deploy.
- It integrates with pull requests to block merges when checks fail.

Typical uses

Test on each PR, lint code, build docs, run security checks, and deploy on tags.

CodeQL is GitHub's semantic code analysis engine for finding vulnerabilities and bugs.

- Build a code database and query it with security-focused rules.
- Detects classes of issues that pattern-based linters can miss.
- Integrates with GitHub code scanning and pull request checks.
- Supports multiple languages (for example C/C++, Java, JavaScript, Python, and C#).

Reference

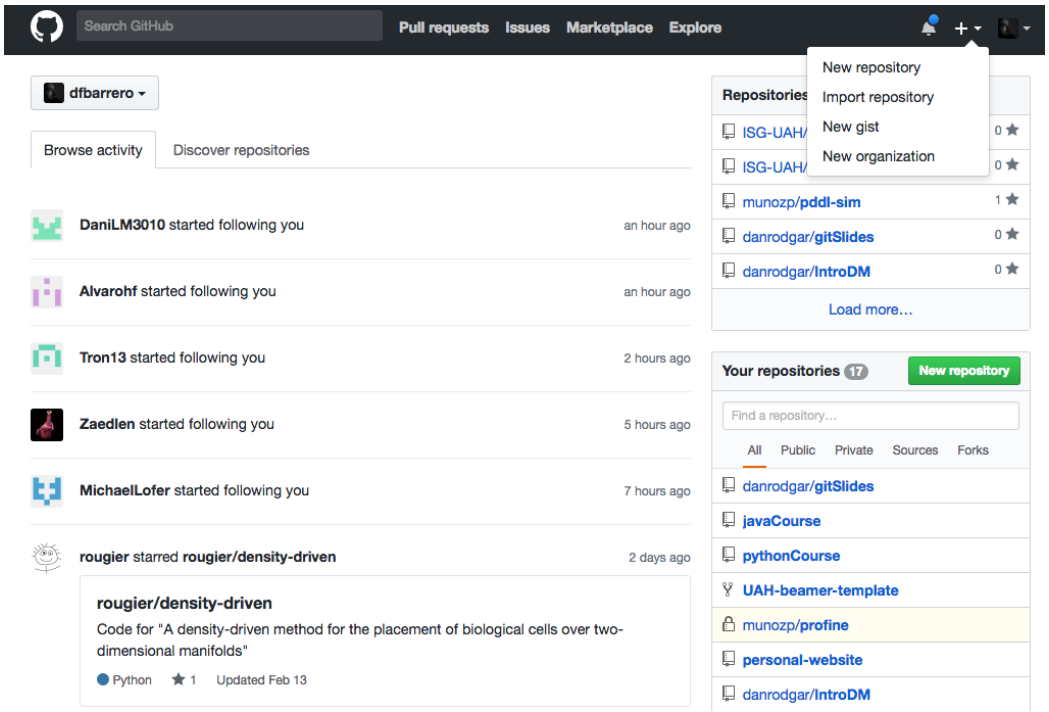
`https://codeql.github.com/`

GitHub

Repository Creation

When creating a repository, you normally configure:

- Name
- Description
- Visibility
- Initial `README.md`
- Optional `.gitignore` and license files



GitHub

README

Special file: `README.md`

- Explains what the project is and how to use it.
- Is automatically rendered by GitHub.
- Usually includes installation, usage, and contribution instructions.
- The `md` extension stands for Markdown.

Markdown is a lightweight markup language.

- Simple to write
- Easy to read in plain text
- Powerful enough for documentation, READMEs, and web content

Typical elements include headings, emphasis, lists, links, images, and code blocks.

Markdown example

```
# I am a header
## I am a subheader

Regular, *italic* and **bold**

- List item 1
- List item 2

[I am a link](http://foo.com)

![I am a pic](markdown.png)

~~~c
printf("Hello, world");
~~~
```

GitHub Pages integrates simple web publishing into the GitHub workflow.

- Create project web sites and documentation portals.
- Publish from a dedicated branch or from a folder such as `docs`.
- It is commonly used together with Markdown-based content.

Typical project URL:
`https://<username>.github.io/<repository>`